AIID Research Notebook

Technical Reference

For Developers, Data Scientists, and Engineers

Version 1.0

TAI Hackathon — Problem Statement 5

Data: AI Incident Database (CC BY-SA 4.0)

Document generated: February 28, 2026

Contents

1 Architecture Overview **2**
1.1 Runtime Requirements . . . 2

2 Data Pipeline **3**
2.1 Section 0 — Setup . . . 3
2.2 Section 1 — Data Extraction . . . 3
2.3 Section 2 — Data Loading . . . 3
2.4 Section 3 — Cleaning & Preprocessing . . . 4
2.4.1 Entity Alias Map . . . 4
2.4.2 Incidents Cleaning . . . 4
2.4.3 Reports Cleaning . . . 4
2.4.4 Report Statistics Merge . . . 5
2.4.5 Classification Cleaning . . . 5
2.4.6 Master DataFrame Build . . . 5
2.4.7 Entity Explosion . . . 5

3 Interactive Filters (Section 4) **6**

4 Visualization Catalog (Section 5) **6**

5 Gradio Web Application (Section 6) **7**

6 Assumptions & Limitations **8**
6.1 Dataset Limitations . . . 8

7 Code Cell Map **9**

8 Reproducibility Checklist **10**

9 Operational Manual: AI Incident Database Explorer **10**
9.1 I. Incident Explorer: Primary Data Retrieval . . . 10
9.2 II. Trend Analysis: Longitudinal Visualization . . . 11
9.3 III. Entity Deep Dive: Stakeholder Analysis . . . 11
9.4 IV. Harm Landscape: Macro-Systemic Overview . . . 11
9.5 V. High Impact Incidents: Editorial Significance . . . 11
9.6 VI. AI Assistant: Generative Synthesis . . . 12

10 Data Integrity Statement **12**

1 Architecture Overview

The notebook is a single-file, end-to-end pipeline that:

1. Scrapes and downloads the latest AIID MongoDB snapshot (`.tar.bz2`).
2. Loads CSV/BSON tables into pandas DataFrames.
3. Cleans, normalizes, and joins five source tables into one master DataFrame.
4. Exposes interactive filters via `ipywidgets`.
5. Renders 12 Plotly/Matplotlib visualizations from the filtered data.
6. Launches a Gradio web application with six tabs *(Incident Explorer, Trend Analysis, Entity Deep Dive, Harm Landscape, High Impact Incidents, and AI Assistant).*

The notebook contains 79 cells total (31 executable code cells, the rest are markdown documentation). This reference covers every code cell.

1.1 Runtime Requirements

The notebook is designed for **Google Colab with a T4 GPU** (available on the free tier). The T4 is required only for the Gradio web application's AI chatbot; all other cells run on CPU.

Packages are installed without version pinning. The table below lists each dependency and the version resolved on Colab at the time of authoring.

Dependency	Tested Version	Purpose
Python	latest	Runtime (Colab default)
requests	latest	HTTP downloads (snapshot fetch)
beautifulsoup4	latest	HTML parsing (snapshot page scraping)
pandas	latest	DataFrame manipulation
numpy	latest	Numerical arrays
plotly	latest	Interactive charts (11 of 12 visualizations)
matplotlib	latest	Word cloud rendering (Chart 11)
gradio	latest	Web UI framework (Section 6)
bson	latest	MongoDB binary format fallback loader
wordcloud	latest	Term frequency visualization
ipywidgets	latest	In-notebook filter controls
tqdm	latest	Progress bars
country_converter	latest	ISO-3166 alpha-3 to country name mapping

2 Data Pipeline

2.1 Section 0 — Setup

Cell 3 — pip install
Installs all packages silently (`-q` flag). Idempotent — safe to re-run. No version pins.

Cell 5 — imports
Loads all modules into the Python namespace. Calls `warnings.filterwarnings("ignore")` to suppress `FutureWarning` and `UserWarning`. Prints `"All imports successful"` on completion.

2.2 Section 1 — Data Extraction

Cell 8 — `get_latest_snapshot_url()`
Scrapes https://incidentdatabase.ai/research/snapshots/, parses all `<a>` tags with `.tar.bz2` hrefs, sorts by the embedded timestamp in the filename, and returns the most recent URL.

Cell 10 — Download
Streams the file with `requests.get(stream=True)` and writes to disk with a `tqdm` progress bar. Skips if the file already exists (size check only, no checksum).

Cell 12 — Extract and locate data root
Extracts with `tarfile.open()` in `r:bz2` mode. The `find_data_root()` function walks the extracted tree looking for `incidents.csv` or a directory named `aiidprod/`. Sets the global `DATA_ROOT` path.

2.3 Section 2 — Data Loading

Cell 15 — `parse_list_field()`
Defines a utility function that converts stringified Python/JSON lists stored in CSV cells into actual Python lists. For example, the string `"['Google', 'Meta']"` becomes the list `['Google', 'Meta']`. It attempts `ast.literal_eval` first, then `json.loads`, then a regex fallback for single-quoted items. Returns an empty list for NaN or empty strings. This function is a critical dependency for the entire entity analysis pipeline — if it fails to parse a value, entities in that row are silently dropped from downstream counts.

Cell 17 — `_load_table()`
Defines the table-loading function. Tries CSV first (faster, columnar), falls back to BSON (MongoDB binary format) if CSV is not found. Also defines `load_bson_file()` and `bson_to_df()` helpers for the BSON path.

Cell 19 — Load all tables
Calls the loading functions to populate five global DataFrames. Also defines `load_classification()` which searches for classification CSVs by name hint and filters BSON records by namespace.

Five tables are loaded:

Variable	Source File	Schema Notes
`incidents_raw`	`incidents.csv`	One row per incident. Contains `incident_id`, `title`, `description`, `date`, plus stringified Python lists for deployers, developers, and harmed parties.
`reports_raw`	`reports.csv`	One row per report. Linked via `incident_id`. Contains `report_number`, `url`, `source_domain`, `date_published` (ISO-8601 with UTC offset).
`mit_raw`	`*mit*.csv` or BSON	MIT AI Risk Repository annotations. Namespace: `"CSETv1_Classifier"`. Fields: `risk_domain`, `risk_subdomain`, `intent`, `timing`, `entity_type`. Coverage: ∼91%.
`gmf_raw`	`*gmf*.csv` or BSON	Goals–Methods–Failures taxonomy. Fields: `Known AI Goal`, `Known AI Technology`, `Known AI Technical Failure`. Coverage: ∼24%.
`cset_raw`	`*cset*.csv` or BSON	CSET annotations. Fields: `Sector of Deployment`, `Location Country`, `Tangible Harm`, `Lives Lost`, `Injuries`. Coverage: ∼36%.

2.4 Section 3 — Cleaning & Preprocessing

2.4.1 Entity Alias Map

Cell 22

Defines a static Python dict mapping 15+ known aliases to canonical names (e.g., `"facebook"` → `"meta"`, `"alphabet"` → `"google"`). Applied via `.str.lower().map()`, non-matches pass through unchanged.

2.4.2 Incidents Cleaning

Cell 24

Cleans the incidents table:

- Renames columns via a case-insensitive alias map to standardized names (`deployers_raw`, `developers_raw`, `harmed_raw`).
- Parses `date` to `datetime64` with `errors="coerce"`.
- Applies `parse_list_field()` to all list-valued columns.
- Normalizes entity names (lowercase, strip whitespace, apply alias map).
- Derives `year = date.dt.year`.

2.4.3 Reports Cleaning

Cell 26

Cleans the reports table:

- Drops `_id` (MongoDB internal).
- Parses `date_published`, `date_submitted`, `date_downloaded` to `datetime64` with `utc=True`.
- Extracts `source_domain` from the report URL.

2.4.4 Report Statistics Merge

Cell 28

Groups `reports_df` by `incident_id` and computes:

Column	Computation
`report_count`	`count(report_number)`
`first_report_date`	`min(date_published)`
`last_report_date`	`max(date_published)`
`report_span_days`	`(last - first).dt.days`

Left-joined back onto `incidents_clean`.

2.4.5 Classification Cleaning

Cell 30 — `clean_cls()`

Standardizes each classification DataFrame:

- Detects the incident ID column via case-insensitive regex and renames to `incident_id`.
- Replaces sentinel values (`"-"`, `"Unknown"`, etc.) with `NaN`.
- Applies entity alias normalization where applicable.

2.4.6 Master DataFrame Build

Cell 32

Sequential left-joins merge the three classification DataFrames onto `incidents_clean`:

```
incidents_df = incidents_clean
    <- LEFT JOIN mit_clean   ON incident_id  (risk_domain, risk_subdomain, ...)
    <- LEFT JOIN gmf_clean   ON incident_id  (ai_goal, ai_failure, ...)
    <- LEFT JOIN cset_clean  ON incident_id  (Sector of Deployment, Lives Lost, ...)
```

Row count is preserved (no incident is duplicated). GMF's `Known AI Goal` is split: the first token becomes `ai_goal` (for 1:1 joins), and the full list is retained in `ai_goals`.

Note: If a classification table was empty at load time — because the corresponding CSV or BSON file was missing or not present in that particular snapshot — the left-join adds no columns from that taxonomy. Downstream chart cells check for column existence before rendering (e.g., `if "risk_domain" in filtered_df.columns`), so missing taxonomies cause charts to be skipped gracefully rather than producing errors.

2.4.7 Entity Explosion

Cell 34

`explode_entity()` creates long-form tables from the list-valued columns:

```
deployers_df  = explode(incidents_df, "deployers_raw",  "deployer")
developers_df = explode(incidents_df, "developers_raw", "developer")
harmed_df     = explode(incidents_df, "harmed_raw",     "harmed_party")
```

Each output has columns: `incident_id`, `date`, `year`, `<entity_col>`. Entity names are lowercased with alias normalization applied.

3 Interactive Filters (Section 4)

Cell 37

Builds `ipywidgets` dropdowns and an `IntRangeSlider` for year. The callback function `apply_filters()` writes to the global `filtered_df`. Filter options are pre-computed from `incidents_df`.

Widget	Type	Filtering Logic
Year range	`IntRangeSlider`	`year.between(lo, hi)`
Risk domain	`Dropdown`	Exact match on `risk_domain` (or "All")
AI goal	`Dropdown`	Exact match on `ai_goal` (or "All")
Deployer	`Dropdown`	`incident_id.isin(deployers_df[deployers_df.deployer == val])`
Harmed party	`Dropdown`	Analogous to deployer filter

All subsequent visualization cells read from `filtered_df`.

4 Visualization Catalog (Section 5)

Each chart cell checks for required column existence before rendering. If a column is absent — because the corresponding classification file was missing or empty at load time — the cell prints a notice and produces no chart. This is expected behavior, not an error.

#	Cell	Lib.	Chart Type	Data Source
1	40	Plotly	Dual-panel bar + line (year counts + YoY %)	`filtered_df` grouped by `year`
2	42	Plotly	Stacked area (risk domain over time, ≥2010)	`filtered_df` + MIT
3	45	Plotly	Two-level sunburst	`risk_domain` / `risk_subdomain`
4	48	Plotly	Side-by-side horizontal bars	`deployers_df` + `developers_df` (top 20 each)

#	Cell	Lib.	Chart Type	Data Source
5	51	Plotly	Heatmap (top 15 goals × top 15 failures)	GMF fields
6	55	Plotly	Choropleth (Natural Earth)	CSET `country_code` via `country_converter`
7	58	Plotly	Funnel chart (4 stages)	CSET `Tangible Harm`, `Injuries`, `Lives Lost`
8	61	Plotly	Horizontal bar (top 18 sectors)	CSET `Sector of Deployment`
9	64	Plotly	Horizontal bar, color = risk domain	`report_count` (top 15)
10	67	Plotly	Horizontal bar (top 20 harmed parties)	`harmed_df`
11	70	Matpl.	Word cloud (800×400, viridis)	`filtered_df.description`
12	73	Plotly	Grouped bar (pre-2023 vs. 2023+ share %)	MIT `risk_domain`

Note: Charts 2, 3, 5, 6, 7, 8, and 12 are conditionally rendered. They depend on columns from the MIT, GMF, or CSET classification tables and will be skipped if those columns are not present.

5 Gradio Web Application (Section 6)

Cell 75 — Static option pre-computation

Pre-computes all dropdown choices and defines helper functions (`ui_search`, `ui_show_detail`, `ui_trend`, `ui_entity_dive`, etc.). The top 80 deployers, developers, and harmed parties are included by frequency. Also defines `_stats_banner()`, which renders a summary HTML card.

Cell 76 — Build and launch Gradio app

Constructs the five-tab interface and launches with `demo.launch(share=True)`:

Tab	Components and Behavior
Incident Explorer	Left column: keyword search + risk domain / AI goal dropdowns + results table. Right column: `gr.ChatInterface` (placeholder). Bottom: detail HTML on row select.
Trend Analysis	Metric dropdown (8 options) + button → single Plotly chart.

Tab	Components and Behavior
Entity Deep Dive	Entity type dropdown + entity name dropdown (dynamically updated via `.change()`). Outputs: stats HTML, timeline plot, harm-type pie chart, top incidents table.
Harm Landscape	Single "Load" button → sunburst, choropleth, and sector bar charts.
High Impact	Slider (10–50 incidents) → table + bar chart of most-reported incidents.

The public Gradio URL is valid for up to 72 hours. On a **Colab T4 GPU** (free tier), the AI chatbot responds in approximately **2 seconds** per query, with a maximum of around **6 seconds** for complex questions.

6 Assumptions & Limitations

It is important to understand that the limitations stated below all originate from the **upstream dataset** (the AI Incident Database itself) and not the limitations that originate from **design decisions in the notebook code**. The former cannot be resolved by modifying the notebook; they reflect the current state of the AIID data collection and annotation efforts.

6.1 Dataset Limitations

The following limitations are **inherent properties of the AI Incident Database**. They are not caused by this notebook and cannot be resolved by modifying the code.

Area	Detail
Incident definition	The AIID catalogs only incidents that have been *reported and submitted* — primarily those receiving English-language media coverage. Unreported harms, internal company incidents, and incidents in regions with limited English-language press are systematically absent. This is a fundamental characteristic of any submission-based incident registry and cannot be addressed by code changes.
Taxonomy coverage	The three classification systems have different coverage levels: MIT annotates ∼91% of incidents, GMF ∼24%, and CSET ∼36%. These are maintained by independent research teams with different capacity and scope. Charts that depend on a partially-covered taxonomy show only the *annotated subset*, not the full picture. The missing annotations do not exist in the source data; they cannot be inferred or generated by the notebook.

Area	Detail
Geographic bias	The CSET `Location Country` field is sparse and heavily skewed toward English-language journalism. The choropleth map reflects *where incidents are documented*, not where AI harm is most prevalent. Correcting this would require supplementing the database with non-English media sources — a data collection effort, not a code change.
No severity metric	The AIID does not include a standardized severity score. This notebook uses `report_count` as a proxy for significance, but this reflects *media interest*, not actual magnitude of harm. Well-known brands attract disproportionate coverage. A true severity metric would require the AIID to introduce a structured severity rating field.
Multi-value AI goals	The GMF "Known AI Goal" field can contain multiple comma-separated goals. The notebook's `ai_goal` column retains only the first token for clean 1:1 joins; the full list is preserved in `ai_goals` and `goals_df`.

7 Code Cell Map

For quick navigation, the table below lists every executable code cell.

Cell	Purpose
3	`pip install` — install all dependencies
5	Import all libraries; print "All imports successful"
8	Scrape snapshot page; identify latest `.tar.bz2` URL
10	Download snapshot with progress bar
12	Extract archive; locate `DATA_ROOT`
15	Define `parse_list_field()`
17	Define `_load_table()` and BSON helpers
19	Load all 5 tables into global DataFrames
22	Define `ENTITY_ALIASES` dict
24	Clean incidents (rename, parse dates, normalize entities)
26	Clean reports (parse dates, extract domains)
28	Compute per-incident report statistics and merge
30	Define `clean_cls()`; clean MIT, GMF, CSET
32	Build master `incidents_df` via left-joins
34	Explode entity lists into long-form tables
37	Build `ipywidgets` filter controls

Cell	Purpose
40	Chart 1: Incidents per year + rolling average
42	Chart 2: Risk domain stacked area
45	Chart 3: Harm landscape sunburst
48	Chart 4: Top deployers and developers
51	Chart 5: AI goal vs. failure heatmap
55	Chart 6: Geographic choropleth
58	Chart 7: Harm severity funnel
61	Chart 8: Sector of deployment
64	Chart 9: Most-covered incidents
67	Chart 10: Harmed party breakdown
70	Chart 11: Word cloud
73	Chart 12: Emerging risk shift
75	Pre-compute UI options; define all UI helper functions
76	Build and launch Gradio app
78	(Empty cell)

8 Reproducibility Checklist

1. Run all cells top-to-bottom in a fresh runtime (Runtime → Restart and run all).
2. To pin a snapshot: set `filename = "backup-YYYYMMDDHHMMSS.tar.bz2"` and `latest_url` manually before **Cell 10**.
3. All randomness is deterministic (no random seeds are used — all aggregations are counts and joins).
4. The Gradio app's `share=True` generates a temporary public URL valid for 72 hours. Remove `share=True` in production.
5. Consider pinning package versions for exact reproducibility across Colab runtimes.
6. Ensure the Colab runtime is set to **T4 GPU** (Runtime → Change runtime type → T4 GPU) for the Gradio chatbot to function.

9 Operational Manual: AI Incident Database Explorer

This manual provides a detailed technical guide for navigating the AI Incident Database Explorer to analyze AI-related harms documented since the year 1983 till this date.

9.1 I. Incident Explorer: Primary Data Retrieval

The *Incident Explorer* serves as the central engine for filtering the database's total documented incidents.

1. **Configure Filters:** Input specific terms into the **Keyword Search** (e.g., "deepfake") and refine results using the **Risk Domain**, **Deployer**, or **Harmed Party** dropdown menus.
2. **Temporal Selection:** Adjust the **Year From** and **Year To** sliders to isolate incidents within a specific historical window.
3. **Data Generation:** Click **Search** to populate the results table.
4. **Exporting Data:** Before clicking on "Download", click **Export CSV**. Once the server processes the request, navigate to the *Download* footer and click the filename (e.g., `aiid_filtered_export` to save the file.

9.2 II. Trend Analysis: Longitudinal Visualization

This tab converts raw data into visual trends to identify growth patterns in AI harms.

1. **Select Metric:** Open the **Metric** dropdown to choose a data perspective, such as *Incidents per Year*, *Intent Split*, or *Top 5 Deployers Over Time*.
2. **Render Visualization:** Click the **Generate Chart** button.
3. **Interpretation:** Analyze the resulting line or bar graph. Note the 2-year rolling average line (dark blue) for smoothed trend identification.

9.3 III. Entity Deep Dive: Stakeholder Analysis

Use this tab to investigate the specific footprint of a developer, deployer, or victim.

1. **Set Entity Type:** Choose between **Deployer**, **Developer**, or **Harmed Party**.
2. **Identify Entity:** Use the **Select Entity** searchable dropdown to find a specific organization (e.g., "OpenAI" or "Tesla").
3. **Execute Analysis:** Click **Analyze** to generate a custom dashboard featuring an *Incident Timeline* and a *Harm Type Breakdown* pie chart for that specific entity.

9.4 IV. Harm Landscape: Macro-Systemic Overview

This section provides a high-level view of the taxonomy, geography, and industrial sectors affected.

1. **Initialize Data:** Click the **Load Harm Landscape** button to render global charts.
2. **Taxonomy Exploration:** Interact with the *MIT Risk Taxonomy Sunburst* to see how incidents are classified (e.g., 34% Malicious Actors & Misuse).
3. **Geospatial/Sector Analysis:** Review the *Geographic Distribution* map and the *Top 15 Deployment Sectors* bar chart to identify high-risk regions and industries.

9.5 V. High Impact Incidents: Editorial Significance

This tab ranks incidents by their public visibility and media footprint.

1. **Define Scope:** Adjust the **Number of Incidents to show** slider (range: 10 to 50).
2. **Load Metrics:** Click **Load** to generate the leaderboard.
3. **Cross-Domain Comparison:** Use the *Coverage Chart* to compare media reports across different Risk Domains (color-coded by category).

9.6 VI. AI Assistant: Generative Synthesis

The assistant leverages RAG (Retrieval-Augmented Generation) to answer complex research questions.

1. **Query Submission:** Type a natural language question into the **Your question** field.
2. **Calibrate Retrieval:** Adjust the **Retrieved Incidents** slider to determine how many database records the AI should reference (up to 25).
3. **Verification:** Click **Ask**. To verify the accuracy, toggle **Show retrieved incidents** to view the raw source data used for the answer.

10 Data Integrity Statement

This notebook does not generate, fabricate, or modify any data. Every number, category, and label comes directly from the official AI Incident Database. Specifically:

- The data is downloaded from the **official AIID snapshots page**, verified by a date-stamped filename.
- The three classification systems (MIT, GMF, CSET) were created by **independent research teams**, not by the authors of this notebook.
- The only transformations applied are formatting adjustments: standardizing column names, parsing dates, and mapping 15 known company name changes. No incident content is altered.
- Coverage gaps are **explicitly disclosed**: 91%, 24%, and 36% for the three taxonomies respectively.
- The notebook is **fully reproducible**: running it again on the same date produces identical results. Running it on a different date produces results reflecting any new incidents added to the database in the interim.

Data: AI Incident Database — CC BY-SA 4.0.
Built for TAI Hackathon — Problem Statement 5.

Data: AI Incident Database — CC BY-SA 4.0.
Built for TAI Hackathon — Problem Statement 5.